

Ping and Traceroute Homework 2 Report

my_ping.py and my_traceroute.py — Performing basic ping and traceroute actions

Name: Abdul Aleem Shaik Gadda

Course / Section: CSCI 651

Date: 09 October 2026

Helper file(s) used: utils.py

1. Introduction

The programs, `my_ping.py` and `my_traceroute.py`, are command-line network tools built on Python's standard socket library that reimplement the standard ping and traceroute utilities with macOS-style output. The program `my_ping.py` sends ICMP echo request packets over a raw socket, matches each echo reply by identifier and sequence number, and prints per-packet round-trip times along with a closing summary of packet loss and min/avg/max/stddev statistics. The program `my_traceroute.py` sends probes with an increasing TTL, reads the ICMP time exceeded and port unreachable replies, and prints each hop's address and round-trip times, with optional reverse-DNS lookups, a configurable number of probes per hop, and a summary of unanswered probes. Both share a helper module, `utils.py`, which provides the Internet checksum, ICMP packet construction, and IP header parsing. They require root privileges to open raw sockets and are run from the command line, with additional runs using the `-c`, `-i`, `-s`, and `-t` options for ping and the `-n`, `-q`, and `-S` options for traceroute to verify packet matching, timeout handling, and output formatting.

2. Output of the Program for different flags used with `my_ping.py`

The screenshots below show the program `my_ping.py` running with different flags.

2.1 No flag

Command used: `sudo python3 my_ping.py google.com`

Output from Program:

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py google.com
PING google.com (142.250.65.78): 56 data bytes
64 bytes from 142.250.65.78: icmp_seq=0 ttl=117 time=15.262 ms
64 bytes from 142.250.65.78: icmp_seq=1 ttl=117 time=27.134 ms
64 bytes from 142.250.65.78: icmp_seq=2 ttl=117 time=70.653 ms
64 bytes from 142.250.65.78: icmp_seq=3 ttl=117 time=102.745 ms
64 bytes from 142.250.65.78: icmp_seq=4 ttl=117 time=21.269 ms
64 bytes from 142.250.65.78: icmp_seq=5 ttl=117 time=20.086 ms
64 bytes from 142.250.65.78: icmp_seq=6 ttl=117 time=19.159 ms
64 bytes from 142.250.65.78: icmp_seq=7 ttl=117 time=24.979 ms
64 bytes from 142.250.65.78: icmp_seq=8 ttl=117 time=18.762 ms
64 bytes from 142.250.65.78: icmp_seq=9 ttl=117 time=19.081 ms
64 bytes from 142.250.65.78: icmp_seq=10 ttl=117 time=16.846 ms
64 bytes from 142.250.65.78: icmp_seq=11 ttl=117 time=19.231 ms
64 bytes from 142.250.65.78: icmp_seq=12 ttl=117 time=16.037 ms
64 bytes from 142.250.65.78: icmp_seq=13 ttl=117 time=40.162 ms
^C
--- google.com ping statistics ---
14 packets transmitted, 14 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 15.262/30.815/102.745/24.364 ms
```

We can see from the figure that `my_ping` successfully sends and receives ICMP echo packets identified by the last 16 bits of the process ID. Press Ctrl-C on the keyboard to raise an interrupt.

2.2 -c flag used for count

Command used: `sudo python3 my_ping.py -c 4 google.com`

Output from Program:

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py -c 4 google.com
Password:
PING google.com (142.250.65.78): 56 data bytes
64 bytes from 142.250.65.78: icmp_seq=0 ttl=117 time=19.577 ms
64 bytes from 142.250.65.78: icmp_seq=1 ttl=117 time=26.382 ms
64 bytes from 142.250.65.78: icmp_seq=2 ttl=117 time=25.511 ms
64 bytes from 142.250.65.78: icmp_seq=3 ttl=117 time=16.497 ms

--- google.com ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 16.497/21.992/26.382/4.114 ms
```

We can see that the -c flag limits the number of ICMP packets requested and received based on the numerical count provided.

2.3 -t flag is used for timeout, and -i flag is used for waiting time between each request sent

Command used: `sudo python3 my_ping.py -t 5 google.com`

Command used: `sudo python3 my_ping.py -i 2 google.com`

Command used: `sudo python3 my_ping.py -t 10 -i 2 google.com`

Output from Program for flag t

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py -t 5 google.com
PING google.com (142.250.65.78): 56 data bytes
64 bytes from 142.250.65.78: icmp_seq=0 ttl=117 time=16.284 ms
64 bytes from 142.250.65.78: icmp_seq=1 ttl=117 time=19.113 ms
64 bytes from 142.250.65.78: icmp_seq=2 ttl=117 time=15.758 ms
64 bytes from 142.250.65.78: icmp_seq=3 ttl=117 time=19.439 ms
64 bytes from 142.250.65.78: icmp_seq=4 ttl=117 time=18.301 ms

--- google.com ping statistics ---
5 packets transmitted, 5 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 15.758/17.779/19.439/1.492 ms
```

Output from Program for flag i

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py -i 2 google.com
PING google.com (142.250.65.78): 56 data bytes
64 bytes from 142.250.65.78: icmp_seq=0 ttl=117 time=94.092 ms
64 bytes from 142.250.65.78: icmp_seq=1 ttl=117 time=16.760 ms
64 bytes from 142.250.65.78: icmp_seq=2 ttl=117 time=19.650 ms
64 bytes from 142.250.65.78: icmp_seq=3 ttl=117 time=16.689 ms
64 bytes from 142.250.65.78: icmp_seq=4 ttl=117 time=19.296 ms
64 bytes from 142.250.65.78: icmp_seq=5 ttl=117 time=34.659 ms
^C
--- google.com ping statistics ---
6 packets transmitted, 6 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 16.689/33.524/94.092/27.776 ms
```

Output from Program for flag t and i

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py -t 10 -i 2 google.com
PING google.com (142.250.65.78): 56 data bytes
64 bytes from 142.250.65.78: icmp_seq=0 ttl=117 time=17.766 ms
64 bytes from 142.250.65.78: icmp_seq=1 ttl=117 time=95.500 ms
64 bytes from 142.250.65.78: icmp_seq=2 ttl=117 time=18.304 ms
64 bytes from 142.250.65.78: icmp_seq=3 ttl=117 time=18.130 ms
64 bytes from 142.250.65.78: icmp_seq=4 ttl=117 time=19.061 ms

--- google.com ping statistics ---
5 packets transmitted, 5 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 17.766/33.752/95.500/30.877 ms
```

We can see from the pictures that the flag “t” represents the timeout; for example, if t is set to 5 seconds, the packets are transmitted for 5 seconds before the program exits. The default interval between packets of my_ping is 1 second, so we can see 5 packets over 5 seconds. If we use the “i” flag, the interval between packets is set to the specified number of seconds. But the program keeps streaming continuously until a keyboard interrupt is raised.

Combining both the flags has a clear representation that, if the timeout is set to 10 seconds, and the interval time is set to 2 seconds, then the number of packets transmitted and received ideally should be $10/2 = 5$, and we can see from the output that clearly there are only 5 packets transmitted by combining both the flags.

2.4 -s flag for packet size

Command used: `sudo python3 my_ping.py -c 3 -s 128 google.com`

Output from Program:

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py -c 3 -s 128 google.com
Password:
PING google.com (142.250.65.78): 128 data bytes
136 bytes from 142.250.65.78: icmp_seq=0 ttl=117 time=20.654 ms
136 bytes from 142.250.65.78: icmp_seq=1 ttl=117 time=17.666 ms
136 bytes from 142.250.65.78: icmp_seq=2 ttl=117 time=59.611 ms

--- google.com ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 17.666/32.644/59.611/19.108 ms
```

We can see from the picture that the 3 packets are carrying a data payload of 128 bytes, while my_ping shows 136 bytes because the 8 bytes of ICMP header information are added when the packet is constructed. Thus, my_ping reports a total of s+8 bytes.

2.5 What if the hostname is not found or unreachable?

Command used: `sudo python3 my_ping.py -c 5 0.0.0.0 (unreachable host)`

Command used: `sudo python3 my_ping.py -c 5 googlerit.com (host that does not exist)`

Output from Program for unreachable host

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py -c 5 0.0.0.0
Password:
PING 0.0.0.0 (0.0.0.0): 56 data bytes
ping: sendto: Socket is not connected
ping: sendto: Socket is not connected
Request timeout for icmp_seq 0
ping: sendto: Socket is not connected
Request timeout for icmp_seq 1
ping: sendto: Socket is not connected
Request timeout for icmp_seq 2
ping: sendto: Socket is not connected
Request timeout for icmp_seq 3

--- 0.0.0.0 ping statistics ---
5 packets transmitted, 0 packets received, 100.0% packet loss
```

Output from Program for unknown host

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_ping.py -c 5 googlerit.com
ping: cannot resolve googlerit.com: Unknown host
```

We can see from the pictures that if the host is unreachable, my_ping returns an error that says “Socket is not connected,” whereas if the host is unknown or does not exist, it prints “Unknown host.”

3. Output of the Program for different flags used with my_traceroute.py

The screenshots below show the implementation of the program my_traceroute.py with different flags.

3.1 No flag

Command used: sudo python3 my_traceroute.py google.com

Output from Program:

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_traceroute.py google.com
Password:
traceroute to google.com (142.250.65.78), 64 hops max, 40 byte packets
 1 gnat2-vrrp-vlan4018.rit.edu (10.118.255.253) 10.787 ms 6.034 ms 5.256 ms
 2 * 129.21.255.124 (129.21.255.124) 9.583 ms *
 3 rit-east-pp-rtr001-100g-vlan856.net.rit.edu (129.21.8.126) 84.754 ms 10.317 ms 6.254 ms
 4 buf-9208-rit-cdn.nysernet.net (199.109.111.9) 7.208 ms 9.317 ms 9.615 ms
 5 buf-55a1-buf-9208-cdn.nysernet.net (199.109.107.213) 11.642 ms 9.562 ms 7.776 ms
 6 199.109.107.145 (199.109.107.145) 10.548 ms 9.029 ms 9.540 ms
 7 bin-540-rit-540-cdn.nysernet.net (199.109.107.142) 17.818 ms 17.538 ms 16.924 ms
 8 nyc111-57c3-bin-540-cdn.nysernet.net (199.109.107.118) 15.898 ms 16.063 ms 19.274 ms
 9 lclgaa-ax-in-f14.1e100.net (142.250.65.78) 15.813 ms 17.803 ms 17.654 ms
```

We can see that the my_traceroute.py maps out the number of hops that a packet takes before it reaches the destination IP address.

3.2 -n flag

Command used: `sudo python3 my_traceroute.py -n google.com`

Output from Program:

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_traceroute.py -n google.com
traceroute to google.com (142.250.65.78), 64 hops max, 40 byte packets
 1  10.118.255.253  13.899 ms  4.960 ms  5.677 ms
 2  129.21.255.124  8.281 ms  7.417 ms *
 3  129.21.8.126  10.169 ms  7.296 ms  7.252 ms
 4  199.109.111.9  7.792 ms  9.644 ms  10.023 ms
 5  199.109.107.213  9.581 ms  8.006 ms  9.508 ms
 6  199.109.107.145  9.580 ms  9.962 ms  9.605 ms
 7  199.109.107.142  17.179 ms  16.183 ms  16.761 ms
 8  199.109.107.118  15.755 ms  15.167 ms  16.765 ms
 9  142.250.65.78  17.927 ms  110.990 ms  19.394 ms
```

We can see from the picture that the `-n` flag successfully prints hop addresses numerically rather than symbolically.

3.3 -q flag

Command used: `sudo python3 my_traceroute.py -q 1 google.com`

Output from Program:

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_traceroute.py -q 1 google.com
traceroute to google.com (142.250.65.78), 64 hops max, 40 byte packets
 1  gnat2-vrrp-vlan4018.rit.edu (10.118.255.253)  9.275 ms
 2  *
 3  rit-east-pp-rtr001-100g-vlan856.net.rit.edu (129.21.8.126)  15.439 ms
 4  buf-9208-rit-cdn.nysernet.net (199.109.111.9)  123.909 ms
 5  buf-55a1-buf-9208-cdn.nysernet.net (199.109.107.213)  10.212 ms
 6  199.109.107.145 (199.109.107.145)  10.613 ms
 7  bin-540-rit-540-cdn.nysernet.net (199.109.107.142)  16.998 ms
 8  nyc111-57c3-bin-540-cdn.nysernet.net (199.109.107.118)  16.661 ms
 9  lclgaa-ax-in-f14.1e100.net (142.250.65.78)  18.671 ms
```

We can see from the picture that the `-q` flag successfully sets the number of probes per TTL to `n` queries.

3.4 -S flag

Command used: `sudo python3 my_traceroute.py -S google.com`

Output from Program:

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_traceroute.py -S google.com
traceroute to google.com (142.250.65.78), 64 hops max, 40 byte packets
 1 gnat2-vrrp-vlan4018.rit.edu (10.118.255.253) 13.190 ms 6.419 ms 6.361 ms
 2 129.21.255.124 (129.21.255.124) 12.906 ms 18.714 ms *
 3 rit-east-pp-rtr001-100g-vlan856.net.rit.edu (129.21.8.126) 10.503 ms 7.824 ms 7.323 ms
 4 buf-9208-rit-cdn.nysernet.net (199.109.111.9) 9.622 ms 8.520 ms 8.481 ms
 5 buf-55a1-buf-9208-cdn.nysernet.net (199.109.107.213) 14.600 ms 8.121 ms 13.349 ms
 6 199.109.107.145 (199.109.107.145) 9.362 ms 10.728 ms 9.512 ms
 7 bin-540-rit-540-cdn.nysernet.net (199.109.107.142) 15.864 ms 16.480 ms 17.972 ms
 8 nyc111-57c3-bin-540-cdn.nysernet.net (199.109.107.118) 18.519 ms 16.130 ms 16.134 ms
 9 lclgaa-ax-in-f14.1e100.net (142.250.65.78) 15.594 ms 108.210 ms 17.335 ms

Summary of unanswered probes:
Hop  Sent  Unanswered
1    3      0 (0%)
2    3      1 (33%)
3    3      0 (0%)
4    3      0 (0%)
5    3      0 (0%)
6    3      0 (0%)
7    3      0 (0%)
8    3      0 (0%)
9    3      0 (0%)
```

We can see from the picture that the -S (uppercase only) flag successfully prints a summary of how many probes were not answered for each hop.

3.5 What if the hostname is not found or unreachable?

Command used: `sudo python3 my_traceroute.py 0.0.0.0 (unreachable host)`

Command used: `sudo python3 my_traceroute.py googlerit.com (host that does not exist)`

Output from Program for unreachable host

```
(base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_traceroute.py 0.0.0.0
traceroute to 0.0.0.0 (0.0.0.0), 64 hops max, 40 byte packets
traceroute: sendto: No route to host
 1 traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *traceroute: sendto: No route to host
traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *traceroute: sendto: No route to host
traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *
traceroute: sendto: No route to host
 2 traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *traceroute: sendto: No route to host
traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *traceroute: sendto: No route to host
traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *
traceroute: sendto: No route to host
 3 traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *traceroute: sendto: No route to host
traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *traceroute: sendto: No route to host
traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *
traceroute: sendto: No route to host
 4 traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *traceroute: sendto: No route to host
traceroute: wrote 0.0.0.0 40 chars, ret=-1
  *
```

Output from Program for unknown host

```
➤ (base) aleem@Abduls-Macbook-Air-3 HW_2 % sudo python3 my_traceroute.py googlerit.com
googlerit.com: Name or service not known
```

We can see from the pictures that if the host is unreachable, my_traceroute returns an error: “No route to host,” whereas if the host is unknown or does not exist, it prints “Name or Service not Known.”

When using my_traceroute on an unreachable host, the program must be terminated with Ctrl-C so it does not keep waiting until 64 hops (the default maximum) have been tested.

4. Limitations

The programs my_ping and my_traceroute offer basic functionality similar to the native network utilities ping and traceroute.

4.1 my_ping

The my_ping program has some limitations compared to the native ping utility.

1. The native ping utility has native timeout and reply-handling behavior, whereas my_ping has a hard-coded minimum wait of 1 second per reply.
2. Native ping supports the -m flag to set the Time to Live (TTL) of packets before they reach the destination address; my_ping does not support this functionality.

4.2 my_traceroute

The my_traceroute program has some limitations compared to the native traceroute utility.

1. The hardcoded timeout for the probe is 3 seconds, whereas in the native traceroute, it is configurable with the -w flag.
2. Native traceroute also supports flags to filter source addresses, destination ports, maximum TTL, TCP protocol, whereas my_traceroute defaults to UDP probes.

5. Conclusion

The my_ping program reports packet loss and round-trip times. At the same time, the my_traceroute tool identifies intermediate routers by increasing the TTL and receiving ICMP responses, with options to control probes and display results.